



Universidad de Granada

DOBLE GRADO EN INGENIERÍA INFORMÁTICA Y MATEMÁTICAS

METAHEURÍSTICAS

Práctica 3:

Técnicas de Búsqueda basadas en Trayectorias de Poblaciones para el Problema de Asignación Con Restricciones (PAR)

Autor: Jesús Muñoz Velasco

DNI: 76668409W

E-mail: jesusmuve18@correo.ugr.es

Horario prácticas: Martes 15:30 - 17:30 (Grupo A3)

Curso 2025-2026

Índice

1. Descripción del problema	2
2. Aplicación de los algoritmos empleados	3
2.1. Esquema de representación de soluciones	3
2.2. Esquema de representación de restricciones	3
2.2.1. Forma Matricial	3
2.2.2. Forma Vectorial	3
2.3. Criterio de validación de Soluciones	4
2.4. Reparación de soluciones no válidas	4
2.5. Evaluación de una solución (función objetivo)	4
2.5.1. Cálculo de centroides de una solución	4
2.5.2. Cálculo de la desviación de una solución	5
2.5.3. Cálculo de la infactibilidad de una solución	5
2.5.4. Evaluación de una solución	5
3. Estructura del método de búsqueda	6
3.1. Enfriamiento Simulado (ES)	6
3.2. Búsqueda Multiarranque Básica (BMB)	7
3.3. Búsqueda Local Reiterada (ILS-BL)	8
3.4. Hibridación ILS-ES	9
4. Estructura general del código	9
5. Experimentos y análisis de resultados	10
5.1. Descripción de casos del problema	10
5.2. Resultados obtenidos	10
5.2.1. Tablas globales por problema	11
5.2.2. Boxplots globales por problema	13
5.3. Análisis de resultados	15

1. Descripción del problema

El Problema de Agrupamiento con Restricciones (PAR), también conocido como *Constrained Clustering*, consiste en la partición de un conjunto de n datos o instancias en k grupos (clústeres) disjuntos, basándose en la similitud de sus características y cumpliendo en la mayor medida posible un conjunto de restricciones impuestas sobre los pares de instancias.

Formalmente, sea $X = \{x_1, x_2, \dots, x_n\}$ un conjunto de n elementos en un espacio de dimensión d . El objetivo es encontrar una partición $C = \{C_1, C_2, \dots, C_k\}$ tal que se minimice la dispersión intra-clúster (medida como la distancia euclídea de cada elemento al centroide de su clúster), sujeta a un conjunto de restricciones a nivel de instancia (de tipo Must-Link y Cannot-Link).

Hay 2 tipos de restricciones posibles sobre 2 instancias $x_i, x_j \in X$:

-) *Must-Link* (ML). Los elementos x_i y x_j deben pertenecer al mismo clúster.
-) *Cannot-Link* (CL). Los elementos x_i y x_j no pueden pertenecer al mismo clúster.

Dado que encontrar una solución factible que satisfaga todas las restricciones es un problema NP-completo en la mayoría de configuraciones prácticas, relajaremos el incumplimiento de estas condiciones, donde las violaciones se penalizan en la función objetivo. La función objetivo a minimizar se define como

$$fitness(solucion) = desviacion(solucion) + \lambda \cdot infeasibilidad(solucion)$$

donde establecemos $\lambda = \frac{max_{dist}(X)}{restricciones(R)}$ para asegurar que la infactibilidad y la desviación tienen la suficiente relevancia. Además, la desviación de una solución se define como

$$desviacion(C_s) = \frac{1}{k} \sum_{C_i \in C_s} \frac{1}{|C_i|} \sum_{\vec{x}_j \in C_i} \left\| \vec{x}_j - \frac{1}{|C_i|} \sum_{x_j \in C_i} \vec{X}_j \right\|$$

y la infactibilidad como el número de condiciones incumplidas de una solución.

2. Aplicación de los algoritmos empleados

Para la resolución del PAR mediante los algoritmos Aleatorio, Greedy y Búsqueda Local, es necesario establecer un marco de trabajo común que defina cómo se representan, evalúan y manipulan las soluciones.

2.1. Esquema de representación de soluciones

Una solución S se representa como un vector de tamaño n (número de instancias), donde el índice i representa la instancia x_i , y el valor almacenado en $S[i]$ indica el índice del clúster al que ha sido asignada.

$$S = [c_0, c_1, \dots, c_{n-1}] \quad \text{donde} \quad c_i \in \{0, 1, \dots, k-1\}$$

Esta representación garantiza que toda instancia pertenezca a un clúster y solo uno.

2.2. Esquema de representación de restricciones

Para representar el conjunto de restricciones R se pueden usar dos estrategias principalmente, según convenga para mayor eficiencia en las evaluaciones de una solución:

2.2.1. Forma Matricial

El conjunto R se representa como una matriz de dimensión $n \times n$ donde cada elemento R_{ij} representará la restricción existente entre la instancia x_i y la x_j . Podrá tomar los siguientes valores:

-) $R_{ij} = 1$: Indica una restricción *Must-Link*.
-) $R_{ij} = -1$: Indica una restricción *Cannot-Link*.
-) $R_{ij} = 0$: Indica que no existe ninguna restricción entre x_i y x_j .

Por la naturaleza de esta matriz será cuadrada, simétrica y los elementos de la diagonal principal no podrán valer -1 . Es muy útil para saber si 2 elementos tienen alguna restricción entre ellos.

2.2.2. Forma Vectorial

En este caso se ha creado un nuevo tipo de dato llamado “Restricción” y tendrá la siguiente estructura:

```
1 struct Restriccion:
2     i, j          // Índices de los puntos  $x_i$  y  $x_j$  que participan en la restricción
3     tipo          // 1 para Must-Link, -1 para Cannot-Link
```

Código Fuente 1: Estructura de una restricción vectorial

De esta forma, el conjunto de restricciones R será un vector de “Restricciones” según acabamos de describir. Aunque esta estructura sea peor para saber si 2 elementos tienen alguna restricción, es mucho mejor para recorrer las restricciones una por una para cálculos como la infactibilidad.

2.3. Criterio de validación de Soluciones

Dado que el problema establece que hay un número k de clústeres para la solución se considerará que una solución S es válida si y solo si aparecen todos los clústeres en la solución, es decir, si se verifica que

$$c_i \in S \quad \forall c_i \in \{0, 1, \dots, k-1\}$$

2.4. Reparación de soluciones no válidas

El proceso de reparación de una solución S con clústeres vacíos consistirá en seleccionar un elemento $c_0 \in S$ que aparezca al menos 2 veces en el vector (para no dejar dicho clúster vacío) y sustituir su valor por el índice del clúster vacío. Se repite este proceso hasta que no haya clústeres vacíos. Seguiría el siguiente esquema:

```

1  Función RepararSolucion(S):
2      // Inicializamos el vector n_puntos con el número de puntos de cada clúster
   // en la solución S
3      n_puntos = nPuntosClusteres(S)
4
5      Para cada clúster c desde 0 hasta k-1:
6          Si n_puntos[c] == 0:
7              arreglado = Falso
8              instancia = 1
9
10             // Buscar una instancia de un clúster con más de 1 elemento
11             Mientras arreglado == Falso y instancia ≤ n:
12                 cluster_actual = S[instancia]
13
14                 Si n_puntos[cluster_actual] > 1:
15                     // Reasignar al clúster vacío
16                     S[instancia] = c_vacio
17                     n_puntos[cluster_actual] = n_puntos[cluster_actual] - 1
18                     n_puntos[c_vacio] = n_puntos[c_vacio] + 1
19                     arreglado = Verdadero
20
21                 instancia++
22
23     return S

```

Código Fuente 2: Pseudocódigo de la reparación de una solución

2.5. Evaluación de una solución (función objetivo)

La evaluación de una solución requiere calcular los centroides de los clústeres a partir de las asignaciones actuales en S , calcular la desviación media de cada clúster y contabilizar las restricciones violadas consultando la matriz R . Esto se hará de la siguiente forma:

2.5.1. Cálculo de centroides de una solución

```

1  Funcion obtenerCentroides(S):
2      n_puntos = nPuntosClusteres(S)
3
4      // Recorro todos los vectores sumándolos a su cluster
5      Para cada instancia i de X:
6          cluster = S[i]
7          Para cada dimension d de la instancia i:
8              centroides[cluster].coordenada[d] += X[i].coordenada[d]
9

```

```

10
11 // Divido entre el número de cada cluster para calcular la media
12 Para cada cluster c desde 0 hasta k-1
13     Para cada dimension d del centroide centroides[c]:
14         centroides[c].coordenada[d] /= n_puntos[c]
15
16 return centroides

```

Código Fuente 3: Pseudocódigo del cálculo de los centroides de una solución

2.5.2. Cálculo de la desviación de una solución

```

1 Función Desviacion(S):
2     centroides = obtenerCentroides(S)
3     n_puntos = nPuntosClusteres(S)
4
5     // Sumamos las distancias
6     Para cada instancia i de X:
7         cluster = S[i]
8         distanciaic[cluster] += distancia(X[i], centroides[cluster])
9
10    desviacion = 0
11
12    // Sumamos la media de las distancias
13    Para cada clúster c desde 0 hasta k-1
14        Si n_puntos[c] > 0:
15            desviacion += (distanciaic[c]/n)
16
17    return (desviacion/k)

```

Código Fuente 4: Pseudocódigo del cálculo de la desviación de una solución

2.5.3. Cálculo de la infactibilidad de una solución

```

1 Función Infactibilidad(S):
2     inf = 0
3
4     Para cada restricción res en el conjunto de restricciones R
5         juntos = (S[res.i] == S[res.j])
6
7         Si res es de tipo ML y !juntos:
8             inf++ // Error: Deberían estar juntos (Must-Link)
9         Sino si res es de tipo CL y juntos:
10            inf++ // Error: No deberían estar juntos (Cannot-Link)
11
12    return inf

```

Código Fuente 5: Pseudocódigo del cálculo de la infactibilidad de una solución

2.5.4. Evaluación de una solución

Finalmente, usando las funciones previamente descritas, y previamente fijado el valor de lambda, la función objetivo se calculará de la siguiente forma

```

1 Evaluacion(S) {
2     C = Desviacion(S)
3     inf = Infactibilidad(S)
4
5     return (C + lambda * inf)
6 }

```

Código Fuente 6: Pseudocódigo del cálculo de la función objetivo

3. Estructura del método de búsqueda

En esta sección se describe, mediante pseudocódigo, el funcionamiento interno de cada uno de los cuatro algoritmos implementados en esta práctica. Todos ellos comparten la representación de soluciones, la función objetivo y el operador de vecino descritos en la sección anterior.

3.1. Enfriamiento Simulado (ES)

El algoritmo ES parte de una solución inicial aleatoria y permite aceptar soluciones peores con una probabilidad decreciente controlada por la temperatura. A continuación se detalla su estructura completa, reflejando fielmente la implementación realizada.

Cálculo de la temperatura siguiente (esquema de Cauchy modificado)

```
1 Función nextT(beta, T):
2     return T / (1.0 + beta * T)
```

Código Fuente 7: Pseudocódigo del esquema de enfriamiento de Cauchy modificado

Algoritmo principal ES

```
1 Función ES.optimize(problem, maxevals, solucion_inicial):
2     n = problem.getSolutionSize()
3     max_vecinos = 10 * n
4     max_exitos = 1 * n
5     M = maxevals / max_vecinos // número de enfriamientos
6
7     // Inicialización
8     solucion = solucion_inicial
9     Si !problem.isValid(solucion): problem.fix(solucion)
10
11     info = problem.generateFactoringInfo(solucion)
12     evals = 1
13     fitness_actual = info.fitness
14     mejor_solucion = solucion
15     mejor_fitness = fitness_actual
16
17     // Cálculo de temperaturas y beta
18     T0 = (mu * fitness_actual) / (-ln(phi))
19     Tf = 1e-3
20     Si Tf >= T0: Tf = T0 * 1e-3
21     beta = (T0 - Tf) / (M * T0 * Tf)
22     T = T0
23
24     // Dominio de la solución
25     [k_min, k_max] = problem.getSolutionDomainRange()
26
27     // Bucle principal
28     Mientras evals < maxevals:
29         n_exitos = 0
30         n_vecinos = 0
31
32         // Construir lista de movimientos válidos (i, nuevo_cluster)
33         moves = obtenerVecindario(solucion)
34
35         // Barajar aleatoriamente para explorar sin repetición
36         Random.shuffle(moves)
37
38         // --- Bucle interno L(T) ---
```

```

39     Para cada (pos, nuevo_cluster) en moves:
40         Si evals >= maxevals: salir
41         Si n_vecinos >= max_vecinos: salir
42         Si n_exitos >= max_exitos: salir
43
44         // Evaluación incremental del vecino
45         fitness_vecino = problem.fitness(solucion, info, pos, nuevo_cluster)
46         evals++
47         n_vecinos++
48
49         delta_f = fitness_vecino - fitness_actual
50
51         // Criterio de aceptación
52         Si delta_f < 0:
53             aceptar = Verdadero // mejora: siempre aceptar
54         Sino:
55             prob = exp(-delta_f / T)
56             aceptar = (Random.uniforme(0,1) <= prob)
57
58         Si aceptar:
59             solucion.AplicarCambio(pos, nuevo_cluster)
60             n_exitos++
61
62             Si fitness_actual < mejor_fitness:
63                 mejor_fitness = fitness_actual
64                 mejor_solucion = solucion
65
66         // Condición de convergencia: ningún éxito en este enfriamiento
67         Si n_exitos == 0: salir del bucle principal
68
69         // Aplicar esquema de Cauchy
70         T = nextT(beta, T)
71
72     return (mejor_solucion, mejor_fitness, evals)

```

Código Fuente 8: Pseudocódigo del Enfriamiento Simulado (ES)

3.2. Búsqueda Multiarranque Básica (BMB)

La BMB genera `n_arranques` soluciones iniciales aleatorias independientes, aplica la Búsqueda Local (BL) sobre cada una de ellas y retiene la mejor solución global.

```

1  Función BMB.optimize(problem, maxevals):
2      // Parámetros: n_arranques = 5, maxevals_bl = 20000
3      mejor_solucion = vacía
4      mejor_fitness = +∞
5      total_evals = 0
6
7      Para t desde 0 hasta n_arranques - 1:
8          Si total_evals >= maxevals: salir
9
10         // Evaluaciones disponibles para esta ejecución de BL
11         evals_restantes = min(maxevals_bl, maxevals - total_evals)
12
13         // Ejecutar BL con solución inicial aleatoria nueva
14         resultado = BusquedaLocal(problem, evals_restantes)
15         total_evals += resultado.evaluaciones
16
17         // Actualizar mejor global
18         Si resultado.fitness < mejor_fitness:
19             mejor_fitness = resultado.fitness
20             mejor_solucion = resultado.solucion
21
22     return (mejor_solucion, mejor_fitness, total_evals)

```

Código Fuente 9: Pseudocódigo de la Búsqueda Multiarranque Básica (BMB)

3.3. Búsqueda Local Reiterada (ILS-BL)

El esquema ILS combina la BL con un operador de mutación que perturba la mejor solución encontrada para escapar de óptimos locales. La clase base `ILSBase` contiene tanto el operador de mutación como el bucle principal; la subclase `ILSAlgorithm` instancia el algoritmo de optimización interno con la BL.

Operador de mutación

```

1 Función ILSBase.mutate(solucion, k_min, k_max, problem):
2     n = solucion.size()
3     n_mutar = round(p_mutacion * n)    // p_mutacion = 0.2
4     Si n_mutar < 1: n_mutar = 1
5
6     // Permutación aleatoria de índices; tomamos los primeros n_mutar
7     indices = [0, 1, ..., n-1]
8     Random.shuffle(indices)
9
10    k_rango = k_max - k_min + 1
11
12    Para t desde 0 hasta n_mutar - 1:
13        pos = indices[t]
14        cluster_actual = solucion[pos]
15
16        // Elegir un cluster distinto al actual
17        Repetir:
18            nuevo_cluster = Random.entero(k_min, k_max)
19            Hasta que nuevo_cluster != cluster_actual 0 k_rango == 1
20
21        solucion[pos] = nuevo_cluster
22
23    // Reparar si la mutación dejó algún cluster vacío
24    Si !problem.isValid(solucion): problem.fix(solucion)

```

Código Fuente 10: Pseudocódigo del operador de mutación del ILS

Bucle principal ILS

```

1 Función ILSBase.optimize(problem, maxevals):
2     // Parámetros: n_arranques = 5, maxevals_algorithm = 20000, p_mutacion = 0.2
3     [k_min, k_max] = problem.getSolutionDomainRange()
4     total_evals = 0
5
6     // Primera aplicación del algoritmo de optimización (solución aleatoria)
7     evals_restantes = min(maxevals_algorithm, maxevals - total_evals)
8
9     // Optimizamos mediante el algoritmo elegido
10    resultado_inicial = optimizeAlgorithm(problem, evals_restantes)
11    total_evals += resultado_inicial.evaluaciones
12
13    mejor_solucion = resultado_inicial.solucion
14    mejor_fitness = resultado_inicial.fitness
15
16    // Bucle principal: n_arranques - 1 iteraciones adicionales
17    Para t desde 1 hasta n_arranques - 1:
18        Si total_evals >= maxevals: salir
19
20        // Mutar la mejor solución encontrada hasta ahora
21        mutada = copiar(mejor_solucion)
22        mutate(mutada, k_min, k_max, problem)
23
24        // Aplicar el algoritmo de optimización sobre la solución mutada
25        evals_restantes = min(maxevals_algorithm, maxevals - total_evals)
26        // Optimizamos mediante el algoritmo elegido
27        resultado_bl = optimizeAlgorithm(problem, evals_restantes, mutada)

```

```

28         total_evals += resultado_bl.evaluaciones
29
30         // Criterio del mejor: actualizar solo si hay mejora
31         Si resultado_bl.fitness < mejor_fitness:
32             mejor_fitness = resultado_bl.fitness
33             mejor_solucion = resultado_bl.solucion
34
35         return (mejor_solucion, mejor_fitness, total_evals)

```

Código Fuente 11: Pseudocódigo del bucle principal de ILS

En `ILSAlgorithm`, `optimizeAlgorithm` delega en la BL estándar:

```

1 Función ILSAlgorithm.optimizeAlgorithm(problem, maxevals):
2     return BusquedaLocal(problem, maxevals)
3
4 Función ILSAlgorithm.optimizeAlgorithm(problem, maxevals, solucion_inicial):
5     return BusquedaLocal(problem, maxevals, solucion_inicial)

```

Código Fuente 12: Pseudocódigo de ILS-BL

3.4. Hibridación ILS-ES

El algoritmo ILS-ES reutiliza íntegramente el bucle principal de `ILSBase` (sección anterior), sustituyendo únicamente el método de optimización interna: en lugar de la BL se emplea el ES. La clase `ILSESAAlgorithm` sobrescribe `optimizeAlgorithm` delegando en `ESAlgorithm.optimize`.

```

1 Función ILSESAAlgorithm.optimizeAlgorithm(problem, maxevals):
2     return ES.optimize(problem, maxevals)
3
4 Función ILSESAAlgorithm.optimizeAlgorithm(problem, maxevals, solucion_inicial):
5     return ES.optimize(problem, maxevals, solucion_inicial)
6
7 // El bucle principal es idéntico al de ILSBase (véase Código Fuente~\ref{lst:
8 // con n_arranques = 5 y maxevals_algorithm = maxevals / n_arranques.
9 // Los parámetros del ES interno son phi = 0.3 y mu = 0.2.

```

Código Fuente 13: Pseudocódigo de la hibridación ILS-ES

El esquema completo de ILS-ES resulta, por tanto, de sustituir la llamada a BL en el pseudocódigo 11 por la llamada a ES del pseudocódigo 8, manteniendo el mismo operador de mutación (pseudocódigo 10) y el mismo criterio de aceptación del mejor.

4. Estructura general del código

El código está programado sobre la API proporcionada de C++. Los ficheros creados principales se encuentran en la carpeta `inc`. El programa principal se encuentra en el fichero `main.cpp` que se podrá ejecutar de cualquiera de las siguientes formas:

```

1 main
2 main <seed>
3 main <seed> <filename> <ratio> <k>

```

Además se ha creado un script para compilarlo y otro para ejecutarlo, que se deberá ejecutar desde la carpeta raíz del proyecto (la inmediatamente superior a la carpeta que contiene el código). Por tanto, para compilar y ejecutar el proyecto tendremos que ejecutar desde una terminal

```

1 ./compilar.sh
2 ./ejecutar.sh

```

5. Experimentos y análisis de resultados

5.1. Descripción de casos del problema

Esta implementación se ha utilizado para los siguientes casos:

1. Zoo: En donde hay que agrupar animales en función de 16 atributos. Tiene 7 clases ($k = 7$) y 101 instancias.
2. Glass: En donde hay que agrupar distintos vidrios en función de 9 atributos sobre sus componentes químicos. Tiene 7 clases ($k = 7$) y 214 instancias.
3. Bupa: En donde hay que agrupar personas en función de sus hábitos de consumo de alcohol, definido con 5 atributos. Tiene 16 clases ($k = 16$) y 345 instancias.

Más adelante se especificarán el resto de parámetros

5.2. Resultados obtenidos

Se ha ejecutado una batería de pruebas con un programa python (`metrics.py`), ejecutando cada algoritmo para cada *dataset* y *ratio* para 10 semillas distintas, en concreto las siguientes:

```

1 SEEDS = [12345, 67890, 13579, 24680, 98765, 43210, 10293, 84756, 97531, 8642]

```

Código Fuente 14: Semillas utilizadas para las pruebas

Tras ejecutar el programa se han extraído las siguientes métricas:

Algoritmo	Dataset	Ratio	Fitness		Distancia		Incumplimiento		Evals	Tiempo (ms)	
			Media	Std	Media	Std	Media	Std		Media	Std
Greedy	bupa	15	0.2400	0.0024	0.2390	0.0021	0.0007	0.0004	1	264.7	42.42
Greedy	bupa	30	0.2676	0.0025	0.2476	0.0025	0.0133	0.0005	1	551.1	90.79
Greedy	glass	15	0.4880	0.0063	0.4276	0.0039	0.0283	0.0024	1	24.5	3.81
Greedy	glass	30	0.5592	0.0065	0.4259	0.0050	0.0625	0.0015	1	53.1	12.66
Greedy	zoo	15	1.5067	0.0290	1.4973	0.0293	0.0026	0.0014	1	1.8	0.92
Greedy	zoo	30	1.7169	0.0313	1.5996	0.0251	0.0318	0.0030	1	4.7	2.91
Random	bupa	15	0.3284	0.0011	0.2479	0.0015	0.0533	0.0009	100000	1337.7	30.32
Random	bupa	30	0.3335	0.0007	0.2469	0.0020	0.0574	0.0013	100000	2168.3	21.88
Random	glass	15	0.6796	0.0035	0.4289	0.0051	0.1176	0.0028	100000	692.1	17.06
Random	glass	30	0.6968	0.0023	0.4214	0.0037	0.1292	0.0018	100000	993.0	6.78
Random	zoo	15	1.9318	0.0133	1.5668	0.0620	0.0991	0.0164	100000	353.2	47.26
Random	zoo	30	1.9693	0.0197	1.5049	0.0821	0.1261	0.0213	100000	396.1	4.31
BL	bupa	15	0.1702	0.0023	0.1437	0.0028	0.0175	0.0008	82527	765.1	145.81
BL	bupa	30	0.1982	0.0024	0.1437	0.0040	0.0361	0.0014	81769	1383.9	272.22
BL	glass	15	0.4041	0.0063	0.3037	0.0074	0.0471	0.0025	16262	71.6	17.21

Algoritmo	Dataset	Ratio	Fitness		Distancia		Incumplimiento		Evals	Tiempo (ms)	
			Media	Std	Media	Std	Media	Std		Media	Std
BL	glass	30	0.4682	0.0038	0.2862	0.0039	0.0854	0.0028	17931	132.1	32.01
BL	zoo	15	1.0923	0.0368	0.9492	0.0559	0.0389	0.0110	6202	10.1	1.97
BL	zoo	30	1.1968	0.0327	0.8713	0.0642	0.0884	0.0106	6201	14.5	3.63
AM-BEST (Mejor P2)	bupa	15	0.1775	0.0029	0.1473	0.0032	0.0200	0.0009	100000	1259.2	6.92
AM-BEST (Mejor P2)	bupa	30	0.2013	0.0015	0.1451	0.0024	0.0372	0.0019	100000	2075.7	13.71
AM-BEST (Mejor P2)	glass	15	0.4174	0.0050	0.2968	0.0072	0.0566	0.0034	100000	701.4	19.37
AM-BEST (Mejor P2)	glass	30	0.4819	0.0053	0.2804	0.0077	0.0945	0.0022	100000	992.1	3.38
AM-BEST (Mejor P2)	zoo	15	1.1376	0.0656	0.9978	0.0757	0.0380	0.0079	100000	361.1	41.33
AM-BEST (Mejor P2)	zoo	30	1.2600	0.0649	0.9180	0.0969	0.0929	0.0124	100000	404.5	1.78
BMB	bupa	15	0.1786	0.0019	0.1488	0.0029	0.0197	0.0016	100000	1085.5	8.18
BMB	bupa	30	0.2040	0.0008	0.1464	0.0024	0.0382	0.0017	100000	1890.2	20.91
BMB	glass	15	0.3960	0.0045	0.2960	0.0054	0.0469	0.0016	79149	346.4	16.83
BMB	glass	30	0.4620	0.0019	0.2843	0.0042	0.0833	0.0017	84488	635.2	45.08
BMB	zoo	15	1.0338	0.0339	0.8629	0.0555	0.0464	0.0088	29188	49.7	9.65
BMB	zoo	30	1.1655	0.0238	0.8181	0.0579	0.0943	0.0100	30864	74.0	5.94
ILS-BL	bupa	15	0.1734	0.0023	0.1460	0.0029	0.0182	0.0013	100000	978.5	4.20
ILS-BL	bupa	30	0.1986	0.0012	0.1432	0.0028	0.0367	0.0016	100000	1759.2	33.22
ILS-BL	glass	15	0.3897	0.0036	0.2930	0.0057	0.0454	0.0022	58905	249.9	23.21
ILS-BL	glass	30	0.4604	0.0021	0.2829	0.0031	0.0832	0.0022	68315	497.3	36.42
ILS-BL	zoo	15	1.0329	0.0382	0.8704	0.0727	0.0441	0.0118	23883	38.2	5.85
ILS-BL	zoo	30	1.1786	0.0227	0.8438	0.0608	0.0909	0.0110	21931	50.3	7.80
ES	bupa	15	0.2283	0.0027	0.1755	0.0036	0.0350	0.0017	100000	1230.4	7.62
ES	bupa	30	0.2510	0.0018	0.1752	0.0047	0.0502	0.0022	100000	2467.7	95.70
ES	glass	15	0.3999	0.0042	0.2925	0.0040	0.0504	0.0022	100000	494.3	78.52
ES	glass	30	0.4782	0.0028	0.2872	0.0065	0.0896	0.0024	100000	873.9	21.05
ES	zoo	15	0.9848	0.0189	0.8251	0.0165	0.0434	0.0062	67099	102.9	27.31
ES	zoo	30	1.1620	0.0069	0.7847	0.0209	0.1024	0.0050	85935	197.5	34.79
ILS-ES	bupa	15	0.2286	0.0027	0.1791	0.0033	0.0328	0.0020	100000	1229.4	21.55
ILS-ES	bupa	30	0.2496	0.0031	0.1750	0.0046	0.0495	0.0022	100000	2426.8	29.20
ILS-ES	glass	15	0.4020	0.0041	0.2918	0.0061	0.0517	0.0028	100000	489.0	46.63
ILS-ES	glass	30	0.4799	0.0032	0.2830	0.0056	0.0924	0.0024	100000	866.9	3.84
ILS-ES	zoo	15	0.9701	0.0116	0.7632	0.0519	0.0562	0.0121	89888	139.7	15.85
ILS-ES	zoo	30	1.1462	0.0117	0.7705	0.0190	0.1020	0.0079	92319	212.0	13.35

Tabla 1: Resumen de métricas de rendimiento por algoritmo, dataset y ratio.

5.2.1. Tablas globales por problema

Con el objetivo de estudiar mejor este resultado rellenamos también las siguientes tablas con la información extraída de la tabla anterior

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	1.5067	1.4973	0.0026	1.0	1.8
Random	1.9318	1.5668	0.0991	100000.0	353.2
BL	1.0923	0.9492	0.0389	6201.6	10.1
AM-BEST (Mejor P2)	1.1376	0.9978	0.0380	100000.0	361.1
BMB	1.0338	0.8629	0.0464	29188.5	49.7
ILS-BL	1.0329	0.8704	0.0441	23883.4	38.2
ES	0.9848	0.8251	0.0434	67098.9	102.9
ILS-ES	0.9701	0.7632	0.0562	89887.9	139.7

Tabla 2: Resultados obtenidos para el dataset **Zoo** con 15 % de restricciones en 10 ejecuciones independientes (k=7).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	1.7169	1.5996	0.0318	1.0	4.7
Random	1.9693	1.5049	0.1261	100000.0	396.1
BL	1.1968	0.8713	0.0884	6201.2	14.5
AM-BEST (Mejor P2)	1.2600	0.9180	0.0929	100000.0	404.5
BMB	1.1655	0.8181	0.0943	30863.5	74.0
ILS-BL	1.1786	0.8438	0.0909	21930.6	50.3
ES	1.1620	0.7847	0.1024	85935.3	197.5
ILS-ES	1.1462	0.7705	0.1020	92319.2	212.0

Tabla 3: Resultados obtenidos para el dataset **Zoo** con 30 % de restricciones en 10 ejecuciones independientes (k=7).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	0.4880	0.4276	0.0283	1.0	24.5
Random	0.6796	0.4289	0.1176	100000.0	692.1
BL	0.4041	0.3037	0.0471	16262.5	71.6
AM-BEST (Mejor P2)	0.4174	0.2968	0.0566	100000.0	701.4
BMB	0.3960	0.2960	0.0469	79148.9	346.4
ILS-BL	0.3897	0.2930	0.0454	58905.2	249.9
ES	0.3999	0.2925	0.0504	100000.0	494.3
ILS-ES	0.4020	0.2918	0.0517	100000.0	489.0

Tabla 4: Resultados obtenidos para el dataset **Glass** con 15 % de restricciones en 10 ejecuciones independientes (k=7).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	0.5592	0.4259	0.0625	1.0	53.1
Random	0.6968	0.4214	0.1292	100000.0	993.0
BL	0.4682	0.2862	0.0854	17931.4	132.1
AM-BEST (Mejor P2)	0.4819	0.2804	0.0945	100000.0	992.1
BMB	0.4620	0.2843	0.0833	84487.6	635.2
ILS-BL	0.4604	0.2829	0.0832	68314.6	497.3
ES	0.4782	0.2872	0.0896	100000.0	873.9
ILS-ES	0.4799	0.2830	0.0924	100000.0	866.9

Tabla 5: Resultados obtenidos para el dataset **Glass** con 30 % de restricciones en 10 ejecuciones independientes (k=7).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	0.2400	0.2390	0.0007	1.0	264.7
Random	0.3284	0.2479	0.0533	100000.0	1337.7
BL	0.1702	0.1437	0.0175	82526.6	765.1
AM-BEST (Mejor P2)	0.1775	0.1473	0.0200	100000.0	1259.2
BMB	0.1786	0.1488	0.0197	100000.0	1085.5
ILS-BL	0.1734	0.1460	0.0182	100000.0	978.5
ES	0.2283	0.1755	0.0350	100000.0	1230.4
ILS-ES	0.2286	0.1791	0.0328	100000.0	1229.4

Tabla 6: Resultados obtenidos para el dataset **Bupa** con 15 % de restricciones en 10 ejecuciones independientes (k=16).

Algoritmo	Fitness	Distancia	Incumplimiento	Evaluaciones	Tiempo (ms)
Greedy	0.2676	0.2476	0.0133	1.0	551.1
Random	0.3335	0.2469	0.0574	100000.0	2168.3
BL	0.1982	0.1437	0.0361	81769.3	1383.9
AM-BEST (Mejor P2)	0.2013	0.1451	0.0372	100000.0	2075.7
BMB	0.2040	0.1464	0.0382	100000.0	1890.2
ILS-BL	0.1986	0.1432	0.0367	100000.0	1759.2
ES	0.2510	0.1752	0.0502	100000.0	2467.7
ILS-ES	0.2496	0.1750	0.0495	100000.0	2426.8

Tabla 7: Resultados obtenidos para el dataset **Bupa** con 30 % de restricciones en 10 ejecuciones independientes (k=16).

5.2.2. Boxplots globales por problema

Adicionalmente se han generado los siguientes gráficos (script `graphics.py`) para visualizar la calidad de cada algoritmo:

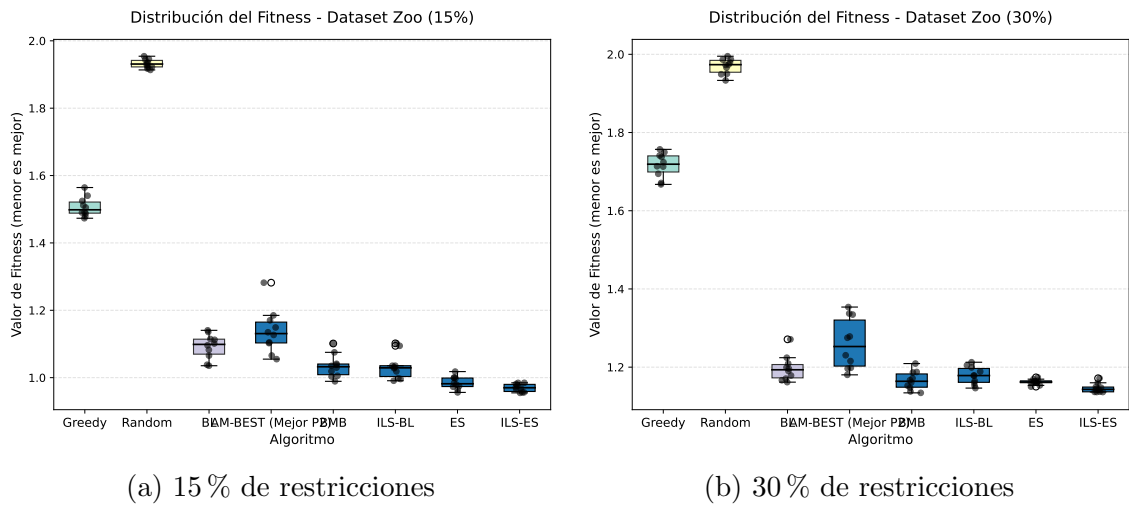


Figura 1: Distribución de los valores de Fitness obtenidos para el dataset **Zoo** en 10 ejecuciones independientes.

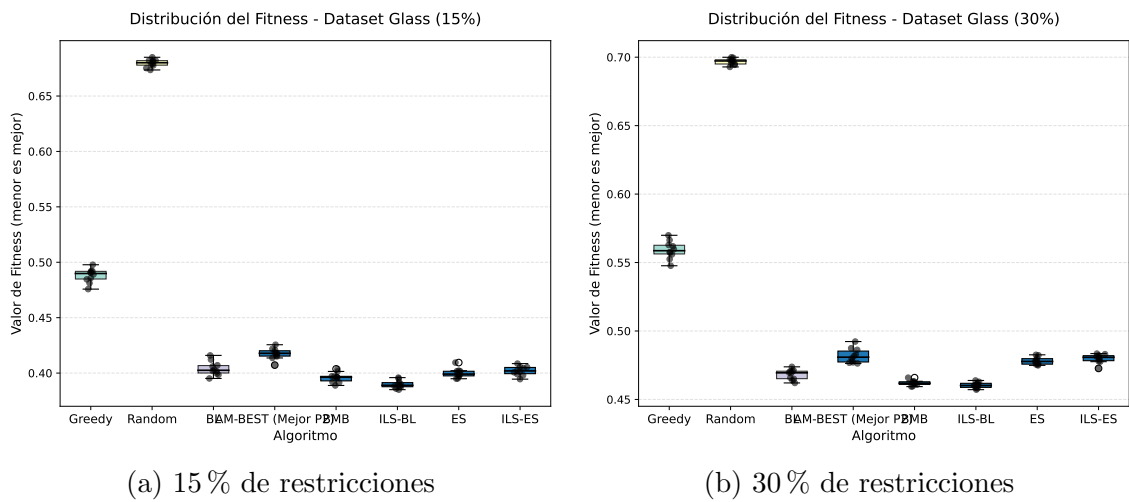


Figura 2: Distribución de los valores de Fitness obtenidos para el dataset **Glass** en 10 ejecuciones independientes.

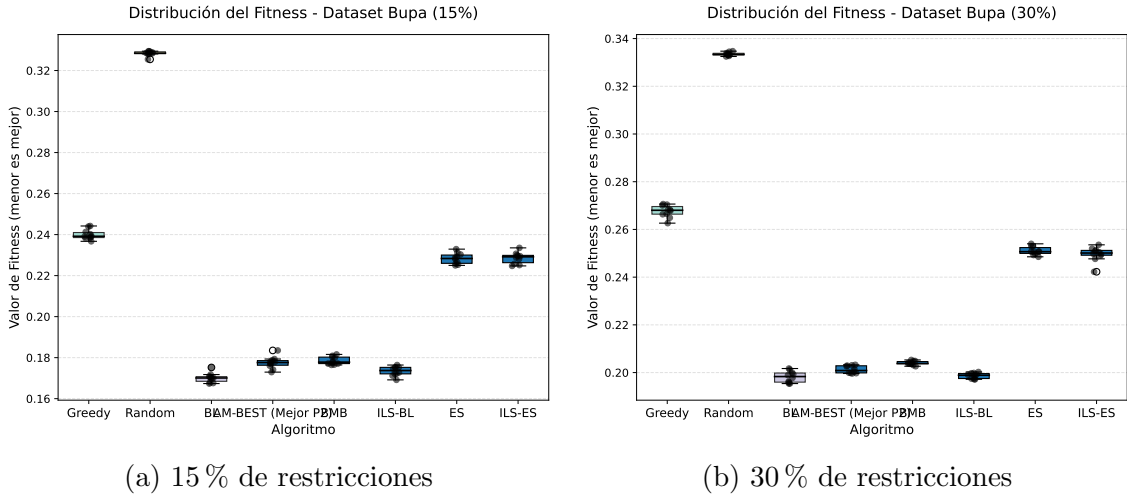


Figura 3: Distribución de los valores de Fitness obtenidos para el dataset **Bupa** en 10 ejecuciones independientes.

5.3. Análisis de resultados

En esta sección se analizan en profundidad los resultados recogidos en las tablas y boxplots de las secciones anteriores. El análisis se organiza comparando, en primer lugar, algoritmos con mayor similitud estructural, extrayendo finalmente conclusiones sobre el comportamiento global.

Observaciones preliminares: referencias de calidad

Antes de entrar en las comparaciones algorítmicas conviene establecer el rango de calidad de las soluciones. *Greedy* y *Random* constituyen los extremos del espectro: *Greedy* genera soluciones de calidad aceptable en tiempo despreciable (≈ 1 evaluación), mientras que *Random*, a pesar de consumir las 100 000 evaluaciones completas, produce sistemáticamente las peores soluciones en los tres datasets. Esto refleja que la simple generación aleatoria masiva, sin ningún mecanismo de mejora, no compensa la ausencia de búsqueda dirigida.

La Búsqueda Local (*BL*) supone ya un salto cualitativo muy notable: reduce el fitness de *Greedy* en torno a un 30 % en Zoo, un 17 % en Glass y un 29 % en Bupa (restricciones al 15 %), empleando además muchas menos evaluaciones que los algoritmos complejos. Este hecho convierte a la *BL* en la referencia fundamental para evaluar el resto de técnicas (parece que por ello se ha hecho hincapié en el guion para analizar este algoritmo en concreto).

Influencia del número de arranques: BL vs. BMB

La *BMB* y la *BL* son los algoritmos más directamente comparables: ambos aplican la misma Búsqueda Local, pero la *BMB* la lanza desde 5 puntos de partida independientes, repartiendo las 100 000 evaluaciones en bloques de hasta 20 000.

- En Zoo (15 %), la *BMB* mejora claramente a la *BL*: el fitness pasa de 1.0923 a 1.0338, una reducción del 5.4 %. El comportamiento se replica al 30 % (de

1.1968 a 1.1655). Los boxplots muestran además una caja más compacta en BMB, lo que indica mayor estabilidad entre ejecuciones.

-) En Glass, la mejora existe pero es menos notable: de 0.4041 a 0.3960 (15 %) y de 0.4682 a 0.4620 (30 %). Los patrones de fitness de Glass, con numerosos óptimos locales de calidad similar, hace que explorar diferentes zonas del espacio de búsqueda no aporte tanto como en Zoo.
-) En Bupa, *BMB* iguala o incluso empeora ligeramente a la *BL*: los valores son prácticamente idénticos (0.1786 vs. 0.1702 al 15 %). Bupa es el dataset más grande ($n = 345$, $k = 16$), lo que implica que la *BL* necesita muchas más evaluaciones para converger (más de 80 000). Al dividir las evaluaciones en 5 bloques de 20 000, ninguna *BL* individual tiene tiempo suficiente para alcanzar el mismo nivel de optimización que una única *BL* con presupuesto completo. Este es un ejemplo claro de la trampa del límite de evaluaciones dividido en algoritmos multiarranque.

En conclusión, la *BMB* supera a la *BL* cuando las instancias son pequeñas o medianas (Zoo, Glass), aprovechando la diversidad de puntos de partida. Sin embargo, en instancias grandes donde la *BL* requiere muchas evaluaciones para madurar, el número de evaluaciones dividido penaliza a la *BMB*.

Importancia de la memoria: BMB vs. ILS-BL

La *ILS-BL* y la *BMB* son estructuralmente muy parecidas: ambas lanzan 5 aplicaciones de la *BL* con un presupuesto de 20 000 evaluaciones cada una. La única diferencia reside en cómo se genera el punto de partida de cada *BL*: la *BMB* lo genera aleatoriamente y de forma independiente, mientras que la *ILS-BL* muta la mejor solución encontrada hasta el momento.

-) En Zoo, los resultados de *ILS-BL* y *BMB* son casi idénticos (1.0329 vs. 1.0338 al 15 %; 1.1786 vs. 1.1655 al 30 %). La pequeña ventaja de *BMB* al 30 % podría indicar que en ese escenario la mayor diversidad de arranques aleatorios es ligeramente más beneficiosa que la explotación guiada por mutación.
-) En Glass, la *ILS-BL* aventaja a la *BMB* de forma más clara (0.3897 vs. 0.3960 al 15 %). Aquí el operador de mutación, que cambia solo el 20 % de los genes, permite explorar vecindades prometedoras sin alejarse demasiado del óptimo local previo.
-) En Bupa la ventaja de *ILS-BL* sobre *BMB* es visible: 0.1734 vs. 0.1786 al 15 % y 0.1986 vs. 0.2040 al 30 %. En este dataset de mayor complejidad, la continuidad proporcionada por la mutación permite que la *BL* parta de soluciones ya parcialmente optimizadas, compensando el presupuesto limitado.

En conjunto, *ILS-BL* es igual o mejor que *BMB* en todos los casos considerados, lo que confirma que reutilizar información de la mejor solución previa mediante una perturbación calibrada es una estrategia más eficiente que reiniciar completamente. Los boxplots respaldan esta conclusión: *ILS-BL* presenta dispersiones similares o menores que *BMB* en todos los datasets.

Exploración del espacio vs. calidad local: BL vs. ES

La comparación entre *BL* y *ES* es especialmente relevante porque ambos algoritmos son de trayectoria única: en lugar de múltiples reinicios, el *ES* intenta superar los óptimos locales aceptando temporalmente soluciones peores.

-) En Zoo, *ES* supera notablemente a la *BL*: 0.9848 vs. 1.0923 al 15 % (mejora del 9.8 %) y 1.1620 vs. 1.1968 al 30 %. La posibilidad de escapar de óptimos locales mediante el criterio de vecindario es especialmente ventajosa en Zoo, donde el espacio de búsqueda tiene barreras locales significativas entre zonas de alta calidad.
-) En Glass, el *ES* no supera a la *BL*: 0.3999 vs. 0.4041 al 15 % (diferencia insignificante) y 0.4782 vs. 0.4682 al 30 %. En Glass, *BL* ya encuentra buenas soluciones con pocas evaluaciones; el tiempo adicional que dedica el *ES* a explorar con temperatura alta no aporta un beneficio neto.
-) En Bupa, el *ES* produce los peores resultados entre los algoritmos avanzados: 0.2283 vs. 0.1702 al 15 %. Este es el resultado más llamativo de toda la práctica. Una explicación plausible es que con $n = 345$ y $k = 16$, el vecindario es enormemente grande ($> 5,000$ movimientos por iteración), y la temperatura inicial T_0 calculada a partir de $\mu \cdot \text{fitness}(S_0)$ puede resultar demasiado elevada, aceptando casi cualquier movimiento en las primeras iteraciones y perdiendo la dirección de mejora. Además, el esquema de Cauchy hace que el enfriamiento sea lento en instancias grandes, dejando pocas evaluaciones para la fase de explotación a baja temperatura.

Los boxplots ilustran además que el *ES* presenta menor dispersión que la *BL* en Zoo (las cajas son más estrechas), mientras que en Bupa la dispersión del *ES* es comparable aunque los valores medios son peores.

Hibridación: ILS-BL vs. ILS-ES

Comparar *ILS-BL* con *ILS-ES* permite aislar el efecto de sustituir la *BL* interna por *ES*.

-) En Zoo, *ILS-ES* obtiene el mejor fitness de todos los algoritmos evaluados: 0.9701 al 15 % y 1.1462 al 30 %, superando a *ILS-BL* (1.0329 y 1.1786). Aquí la combinación de perturbación (mutación del 20 %) con un optimizador que acepta empeoramientos genera una exploración más amplia del espacio de búsqueda.
-) En Glass, *ILS-ES* es ligeramente peor que *ILS-BL* (0.4020 vs. 0.3897 al 15 %; 0.4799 vs. 0.4604 al 30 %). Dado que el *ES* individual ya era comparable a la *BL* en Glass, la degradación introducida por usar *ES* como optimizador interno se amplifica al encadenarlo en el esquema *ILS*.
-) En Bupa, *ILS-ES* es muy similar a *ILS-BL* en términos de fitness medio (0.2286 vs. 0.1734 al 15 %). La desventaja heredada del *ES* en esta instancia se propaga al esquema *ILS*: incluso con el mecanismo de mutación y reencadenamiento,

el *ES* no consigue ajustar la solución tan bien como la *BL* en un espacio de alta dimensionalidad.

En resumen, *ILS-ES* es la mejor opción en datasets con barreras locales pronunciadas (Zoo), pero no generaliza su ventaja a todos los datasets. La elección del optimizador interno en *ILS* es, por tanto, dependiente del problema.

Efecto del porcentaje de restricciones

Al pasar del 15 % al 30 % de restricciones, el fitness de todos los algoritmos aumenta de forma sistemática, dado que la penalización por infactibilidad se vuelve más relevante. Sin embargo, el incremento no es uniforme entre algoritmos:

-) En Zoo, el *ES* y el *ILS-ES* muestran los aumentos más moderados (*ES*: +0,177; *ILS-ES*: +0,176), mientras que la *BL* crece algo más (+0,105). La capacidad del *ES* de aceptar movimientos de empeoramiento le permite explorar zonas factibles que la *BL* no alcanzaría al estar atrapada en un óptimo local infactible.
-) En Bupa, el *ES* y el *ILS-ES* ya eran los peores al 15 % y el incremento hasta el 30 % es similar al del resto de algoritmos, confirmando que el problema principal del *ES* en Bupa es estructural, no específico del nivel de restricciones.

Comparación con el mejor algoritmo de la Práctica 2 (AM-BEST)

El mejor algoritmo de la Práctica 2 (algoritmo memético *AM-BEST*) sirve de referencia de los métodos de búsqueda con poblaciones. Los resultados muestran que:

-) En Zoo, todos los algoritmos de trayectoria de esta práctica (salvo el *ES* y el *ILS-ES*) obtienen fitness similares a *AM-BEST* o ligeramente peores. *ES* e *ILS-ES* son los únicos que superan consistentemente al *AM-BEST*, lo que evidencia la potencia de la aceptación de empeoramientos en instancias con muchas barreras locales.
-) En Glass y Bupa, *AM-BEST* y los algoritmos de esta práctica se sitúan en rangos de fitness muy cercanos. La *BL*, la *BMB* y el *ILS-BL* igualan o superan ligeramente a *AM-BEST* en Glass, mientras que en Bupa el *AM-BEST* es decente pero la *BL* y el *ILS-BL* lo superan.

Análisis del coste computacional

En términos de tiempo de ejecución, se observa que los algoritmos más complejos no siempre son los más costosos:

-) La *BL* es la más rápida entre los algoritmos de optimización (10–765 ms), gracias a su parada temprana cuando no hay mejora.
-) El *ES* y el *ILS-ES* consumen los tiempos más altos en Zoo y Bupa, al agotar sistemáticamente el presupuesto de 100 000 evaluaciones.

-) BMB e *ILS-BL* presentan tiempos intermedios: no siempre agotan el presupuesto porque cada *BL* individual puede converger antes del límite de 20 000 evaluaciones.

La relación calidad/tiempo más favorable corresponde a la *BL* en instancias pequeñas, y a *ILS-BL* o *ILS-ES* cuando se dispone de mayor presupuesto y el dataset tiene estructura compleja (Zoo).

Conclusiones generales

-) La *BL* es una base sólida pero sufre de atrapamiento en óptimos locales, especialmente en instancias con muchos clústeres (Bupa) y complejos (Zoo).
-) La diversificación mejora la *BL* en instancias medianas: tanto *BMB* (diversificación aleatoria) como *ILS-BL* (diversificación guiada) superan a la *BL* en Zoo y Glass. *ILS-BL* tiene ventaja cuando el número de evaluaciones por ejecución es limitado.
-) El *ES* es muy competitivo en datasets complejos (Zoo), donde su capacidad de aceptar empeoramientos le permite superar tanto a la *BL* como a *AM-BEST*. Sin embargo, en espacios de alta dimensionalidad con k grande (Bupa) su rendimiento se degrada notablemente.
-) *ILS-ES* es el mejor algoritmo en Zoo y el mejor o igual en los demás datasets comparado con sus pares de trayectoria. La combinación de perturbación global (mutación) con optimización local capaz de escalar barreras (*ES*) proporciona el mayor poder de exploración-explotación.
-) La complejidad del problema determina qué algoritmo es más adecuado: no existe un algoritmo sistemáticamente mejor que el resto. El tamaño de la instancia, el número de clústeres y la complejidad del esquema de fitness condicionan de forma determinante la elección del método más apropiado.